EcoTrack

Milestone 2 Guide — Carbon Tracking & Goal Management

Week 3 & 4 | Team Reference Document

This document covers four things for the team: (1) prerequisites that must be true before Milestone 2 work starts, (2) the full task list for Milestone 2, (3) the role split — who builds what, why, and with which tools, and (4) a day-by-day flow across the two-week milestone. It follows the same structure as the Milestone 1 Onboarding Guide.

Schema note: the original project spec listed a draft schema (users, profiles, carbon_entries, goals, goal_progress, challenges, notifications, reports) as part of Milestone 1's design task, but marked it incomplete. The team has since finalized an expanded schema (the EcoTrack DBML) that adds emission_factors, carbon_footprint, user_challenges, levels, badges, user_badges, and recommendations. Milestone 2 work below refers to that finalized schema — specifically the carbon_entries, emission_factors, carbon_footprint, goals, and goal_progress tables. Everything else in that schema (challenges, badges, levels, recommendations, profiles) is out of scope for Milestone 2 and deferred to later milestones.

1. Prerequisites — Before Milestone 2 Starts

These five items should be confirmed or completed before anyone begins Milestone 2 feature work. Items 2 and 4 are blocking — nothing else can be built or tested until they're done.

1. Milestone 1 is merged and verified.

- Register and login both tested end-to-end: Postman, JUnit/Mockito controller tests, and the actual Angular UI.
- develop branch is up to date on every teammate's machine (git pull origin develop).

2. schema.sql must be extended (blocking).

- Currently schema.sql only defines roles and users. Before any Milestone 2 entity is written, add CREATE TABLE statements for carbon_entries, emission_factors, carbon_footprint, goals, and goal_progress.
- Field names, types, and constraints must match the finalized EcoTrack DBML exactly — same schema-first discipline used for users/roles (ddl-auto=validate stays in place, so a mismatch fails the app at startup rather than silently drifting).

3. emission_factors needs seed data.

- The calculation engine (amount × factor_value = carbon_emission) has nothing to calculate against until factor rows exist.
- Team should agree on initial factor values per category (transportation, electricity, food, waste, water) before the engine is built — same idea as seeding ROLE_ADMIN/ROLE_USER in Milestone 1.

4. Angular needs a JWT interceptor (blocking for frontend integration).

- auth.ts currently stores the token in memory but nothing attaches it to outgoing requests — that was fine in Milestone 1 because /api/auth/** is public.
- Every Milestone 2 endpoint (carbon entries, goals, dashboard) sits behind JwtAuthFilter and requires an Authorization: Bearer <token> header. An HttpInterceptor that reads the token from Auth.getToken() and attaches it to every outgoing request must exist before any protected call will work from the UI.

5. Keep the Milestone 1 conventions.

- Error responses stay as JSON in the shape { "message": "..." } for every 4xx response — same pattern AuthController established.

- DTOs get @Valid + Bean Validation annotations, with matching @ExceptionHandler(MethodArgumentNotValidException.class) in each new controller.
- Constructor injection throughout; layer boundaries stay the same (controller → service → repository → database).

2. Full Task List for Milestone 2

Per the project spec (Week 3 & 4: Carbon Tracking & Goal Management), plus the two carry-over prerequisites above:

- Extend schema.sql with the five new tables and seed emission_factors (prerequisite, backend).
- Add a JWT HttpInterceptor and route guarding on the frontend (prerequisite, frontend).
- Carbon tracking APIs — log an activity, view history, view a single entry, delete an entry.
- Carbon calculation engine — compute carbon_emission from amount × emission_factor at the time an entry is logged.
- Carbon footprint aggregation — periodic summary (per user, per period) that feeds the dashboard, built from carbon_entries.
- Goal management — create, update, track progress, and mark goals complete.
- Sustainability dashboard — a single summary endpoint plus the Angular UI/charts that consume it.
- Search & filter systems — filter carbon entries and goals by category, date range, and status.
- Tests — JUnit/Mockito for every new service and controller; Jasmine/Karma for every new Angular service.

3. Roles, Responsibilities & Tools

Same three people, continuing the roles they built momentum in during Milestone 1 — Person A stays on the core data layer, Person B moves from security into the calculation/aggregation logic that plays a similar "engine" role JWT did in Milestone 1, and Person C continues owning everything the user sees.

Person A — Carbon Tracking Data Layer

Overall responsibility: owns the carbon_entries and emission_factors tables end to end — entities, repositories, the tracking service, and the REST endpoints the frontend logs activities against.

- Task 1: CarbonEntry.java (entity) — maps to carbon_entries; matches the finalized DBML field-for-field.
- Task 2: EmissionFactor.java (entity) — maps to emission_factors.
- Task 3: CarbonEntryRepository.java — findByUserId, findByUserIdAndCategory, findByUserIdAndActivityDateBetween (needed for the search/filter task).
- Task 4: EmissionFactorRepository.java — findByCategory, used by the calculation engine to look up the right factor.
- Task 5: CarbonEntryService.java — on create, looks up the matching EmissionFactor and computes carbon_emission = amount × factor_value; also exposes history retrieval with category/date filtering.
- Task 6: CarbonEntryController.java — POST /api/carbon-entries, GET /api/carbon-entries (with optional category/from/to query params), GET /api/carbon-entries/{id}, DELETE /api/carbon-entries/{id}.
- Task 7: schema.sql — owns adding the carbon_entries and emission_factors CREATE TABLE statements and the initial factor seed data (Prerequisite 2 and 3 above).

Tools: Java, Spring Boot, Spring Data JPA, MySQL, Postman, JUnit, Mockito

Person B — Goal Management, Footprint Aggregation & Dashboard API

Overall responsibility: builds the DTO contracts (same coordination role as the Milestone 1 DTOs), the goal management domain, and the aggregation logic that turns raw carbon_entries into the numbers the dashboard displays.

- Task 1: CarbonEntryRequest / CarbonEntryResponse, GoalRequest / GoalResponse, GoalProgressResponse (DTOs) — finish these first and share exact field names with Person A immediately, same coordination note as Milestone 1's DTOs.
- Task 2: Goal.java, GoalProgress.java, CarbonFootprint.java (entities) — map to goals, goal_progress, and carbon_footprint.
- Task 3: GoalRepository.java, GoalProgressRepository.java, CarbonFootprintRepository.java.
- Task 4: GoalService.java — create/update a goal, recalculate completion_percentage as new carbon_entries come in, mark a goal completed once its target is reached.
- Task 5: GoalController.java — POST /api/goals, GET /api/goals (with optional status filter), PUT /api/goals/{id}, GET /api/goals/{id}/progress.
- Task 6: CarbonFootprintService.java — aggregates a user's carbon_entries into a periodic carbon_footprint summary (per-category totals + overall total).
- Task 7: DashboardController.java — a single GET /api/dashboard/summary combining footprint totals and goal progress counts, purpose-built for the frontend dashboard so Person C isn't stitching together multiple calls.
- Task 8: Validation — add @Valid + Bean Validation to every new DTO, and the matching @ExceptionHandler(MethodArgumentNotValidException.class) in both new controllers, mirroring AuthController's pattern.

Tools: Java, Spring Boot, Spring Data JPA, MySQL, Postman, JUnit, Mockito

Person C — Frontend (Angular)

Overall responsibility: builds everything the user sees for logging activity, managing goals, and viewing their sustainability dashboard — and, before any of that, makes protected API calls possible at all.

- Task 1 (prerequisite): JWT HttpInterceptor — reads the token from Auth.getToken() and attaches Authorization: Bearer <token> to every outgoing request. Nothing else in this milestone works from the browser without this.
- Task 2: Carbon entry log form (inside carbon-tracking/) — category, amount, unit, and date fields, with basic validation.
- Task 3: Carbon history view — list of past entries with filter controls (category dropdown, date range) wired to Person A's query parameters.
- Task 4: Goal creation form and goal list (inside goals/) — create a goal, view existing goals with a progress bar per goal.
- Task 5: Dashboard component (inside dashboard/) — Chart.js visualizations for footprint overview and goal progress, fed by Person B's /api/dashboard/summary.
- Task 6: carbon-tracking.ts, goals.ts, dashboard.ts (Angular services) — same pattern as auth.ts: one method per endpoint, typed request/response interfaces matching the backend DTOs exactly.
- Task 7: Extend app.routes.ts with /carbon-tracker, /goals, and /dashboard paths, guarded so only an authenticated user can reach them.

Coordination note: Tasks 2, 4, and 5's UI shells can be built immediately with mock data. Task 1 (the interceptor) should be built first regardless, since Task 6's real HTTP calls depend on it — same “build the UI first, wire up real calls once the backend is tested” approach as Milestone 1.

Tools: TypeScript, Angular, Angular Material, RxJS, Chart.js, Jasmine, Karma

4. How the Three Roles' Work Connects

- Person C's forms (carbon entry / goal) submit through the new JWT interceptor, which attaches the token from Milestone 1's login.
- Person B's SecurityConfig-protected routes (everything outside /api/auth/**) let the request through once JwtAuthFilter confirms the token is valid — no changes needed there, it already covers new endpoints automatically.
- The request reaches Person A's CarbonEntryController or Person B's GoalController, which delegate to the matching service.
- Person A's CarbonEntryService looks up the right EmissionFactor and computes carbon_emission, then saves via CarbonEntryRepository.
- Person B's GoalService and CarbonFootprintService read from the same carbon_entries table to update goal progress and dashboard totals.
- Person C's dashboard.ts calls Person B's /api/dashboard/summary and renders the result with Chart.js, closing the loop from activity logged to visual feedback.

5. Day-by-Day Timeline (Two Weeks)

Same format as the Milestone 1 timeline, extended to fit the full two-week window. Adjust pace as needed — the ordering (schema → entities → services → controllers → frontend wiring → tests) is the important part, not the exact day count.

Week 3 — Core Build

Day	Person A	Person B	Person C
Day 1	Extend schema.sql with new tables + seed emission_factors. Create CarbonEntry.java, EmissionFactor.java.	Create DTOs (share fields with A immediately). Create Goal.java, GoalProgress.java, CarbonFootprint.java.	Build JWT HttpInterceptor + route guard. Scaffold carbon-tracking/ and goals/ folders. Build entry log form UI (no backend calls yet).
Day 2	CarbonEntryRepository, EmissionFactorRepository. Start CarbonEntryService (create + calculation).	GoalRepository, GoalProgressRepository, CarbonFootprintRepository. Start GoalService (create/update).	Build goal creation form UI. Build carbon history list UI (mock data).
Day 3	Finish CarbonEntryService. Start CarbonEntryController (POST/GET).	Finish GoalService (progress calc, completion logic). Start GoalController (POST/GET/PUT).	Build dashboard skeleton (Chart.js placeholders). Draft carbon-tracking.ts (stubbed).
Day 4	Finish CarbonEntryController incl. filter query params. Unit tests for CarbonEntryService.	Finish GoalController. Build CarbonFootprintService (aggregation logic). Unit tests for GoalService.	Connect carbon-tracking.ts to A's live endpoints; test create/view entries.
Day 5	Integration test carbon tracking + goals together in Postman with B. Fix issues.	Integration test with A in Postman. Fix issues.	Connect goals.ts to B's endpoints; test create/update/view goals in browser. Push to develop.

Week 4 — Dashboard, Search/Filter, Testing

Day	Person A	Person B	Person C
Day 6	Refine filter/search query support (date range, category). Assist B on aggregation edge cases.	Build GET /api/dashboard/summary (totals, goal progress %, recent entries).	Wire dashboard.ts to /api/dashboard/summary; render live charts.
Day 7	Add edge-case handling for search/filter (empty results, invalid dates).	Add @Valid + exception handling to Goal/CarbonEntry DTOs and controllers.	Build filter/search UI controls (category dropdown, date range picker).
Day 8	Write JUnit/Mockito tests for CarbonEntryController (mirrors AuthControllerTest).	Write JUnit/Mockito tests for GoalController and the dashboard endpoint.	Write Jasmine/Karma tests for carbon-tracking.ts and goals.ts (HttpClientTestingModule).
Day 9	Full end-to-end test with the team: log entry via UI → confirm row + calculated emission in MySQL → create goal → confirm progress updates → dashboard reflects new numbers. Fix bugs.	Same as Person A — shared end-to-end session.	Same as Person A — shared end-to-end session.
Day 10	Final polish, code review, merge PR to develop.	Final polish, code review, merge PR to develop.	Final polish, code review, merge PR to develop. Update guide's "current status" section.

6. Evaluation Criteria (Milestone 2, Week 4)

- Carbon tracking operational — a user can log an activity and see it appear with a correctly calculated carbon_emission.
- Goal management functional — a user can create a goal and watch its progress update as they log entries.
- Sustainability dashboard implemented — the dashboard reflects live totals and goal progress, not placeholder data.
- Search & filter systems working — carbon history and goal lists can be filtered without errors.
- All new services/controllers/Angular services covered by tests, following the Milestone 1 testing pattern.

This document is a living reference — update the "current status" of each section as work progresses, same as the Milestone 1 guide.